

BEGIN

BEGIN
EXCEPTION
END;

...

EXCEPTION;
END;

Podemos crear distintos
bloques dentro del BEGIN → **Bloques Anidados**
para que tenga un exception personalizado

- “Tabla de memoria” → Tablas almacenadas en la memoria, no en la BD.

Store Procedure | Procedimiento

Estructura con nombre, que queda almacenada en la BD. → No como los bloques anónimos
Se compila muchísimas veces menos de lo que se ejecuta.

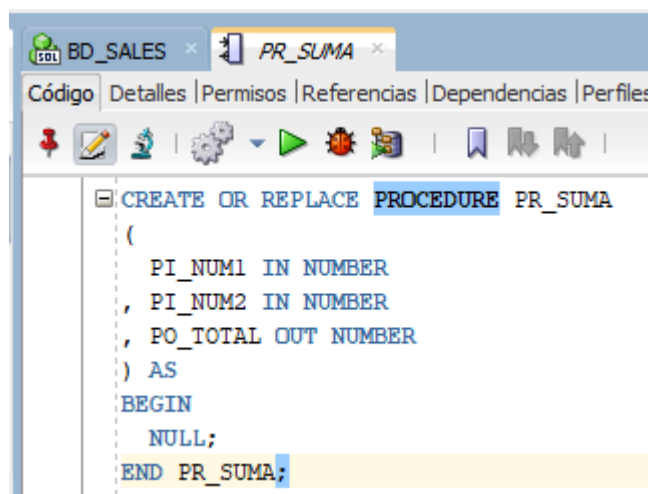
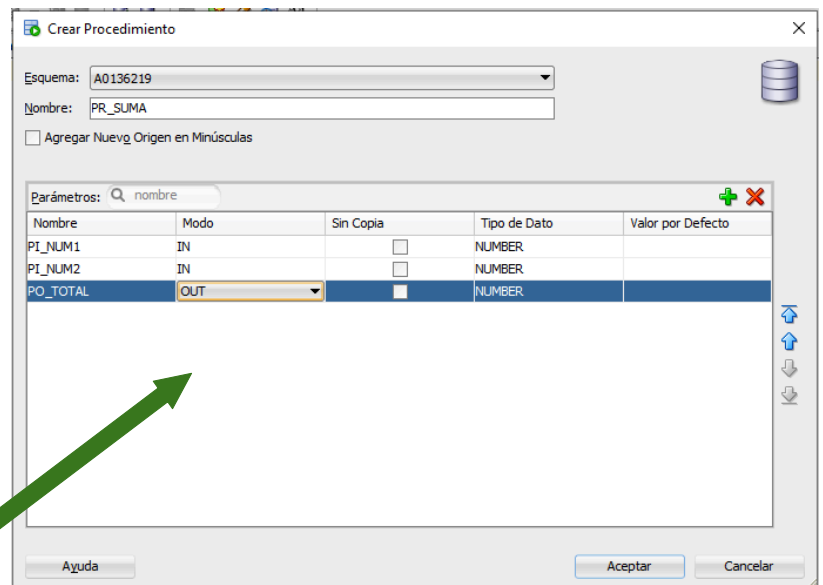
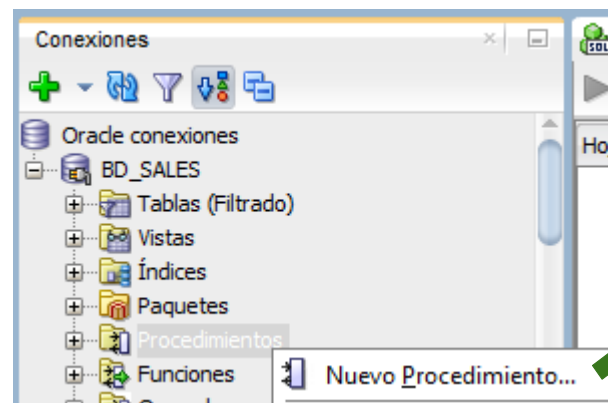
- No puede usar **variables de sustitución**.
- Usa **parámetros**
- No pueden haber 2 procedimientos con el mismo nombre

Estructura: **CREATE OR REPLACE**
PROCEDURE PR_nombre (nombre
<DECLARACIÓN de variables>
BEGIN
...
END;

IN
OUT
IN OUT

Los parámetros pueden ser del tipo:
ENTRADA, SALIDA o ENTRADA-SALIDA
tipo) **IS / AS** → **PARÁMETRO** (PR = Procedi.)

En Oracle SQL Developer:



← Te crea la estructura automáticamente

Utilizamos los procedimientos a partir de los bloques anónimos:

The screenshot shows the SQL Developer interface. On the left, the 'Hoja de Trabajo' (Worksheet) tab contains an anonymous block:

```
SET SERVEROUTPUT ON
DECLARE
  v_tot NUMBER;
BEGIN
  PR_SUMA(1,5,v_tot);
  dbms_output.put_line('Total: '||v_tot);
END;
```

A green arrow labeled 'Bloque Anónimo' points to this code. On the right, the 'PR_SUMA' procedure is defined in the 'Código' (Code) tab:

```
CREATE OR REPLACE PROCEDURE PR_SUMA
(
  PI_NUM1 IN NUMBER
, PI_NUM2 IN NUMBER
, PO_TOTAL OUT NUMBER
) AS
BEGIN
  NULL;
  PO_TOTAL := PI_NUM1 + PI_NUM2;
END PR_SUMA;
```

A green arrow labeled 'Procedimiento' points to this code. Below the worksheet, the 'Salida de Script' (Script Output) window shows the execution results:

```
Tarea terminada en 0,03 segundos

Error que empieza en la línea: 6 del comando :
PR_SUMA
Informe de error -
Comando desconocido

Total: 6

Procedimiento PL/SQL terminado correctamente.
```

Si en el procedimiento asignamos como default un valor a un parámetro, si no le asignamos un nuevo valor en el bloque anónimo, usará el default.

```
CREATE OR REPLACE PROCEDURE PR_MULTIPLICA
(
  PI_NUM1 IN NUMBER
, PI_NUM2 IN NUMBER DEFAULT 2
) AS
  v_result number;
BEGIN
  v_result := pi_num1 * pi_num2;
  dbms_output.put_line('Resultado: '||v_result);
END PR_MULTIPLICA;
```

Si le asignamos 3, hace 5×3

```
DECLARE
  v_tot NUMBER;
BEGIN
  PR_MULTIPLICA(5,3);
END;
```

Salida de Script x

Tarea terminada en 0,028 segundos

Resultado: 15

Procedimiento PL/SQL terminado correctamente.

Si no le asignamos nada, hace $5 \times (\text{default}=2)$

```
DECLARE
  v_tot NUMBER;
BEGIN
  PR_MULTIPLICA(5);
END;
```

Salida de Script x

Tarea terminada en 0,03 segundos

Resultado: 10

Procedimiento PL/SQL terminado correctamente.

The screenshot shows the Oracle SQL Developer environment. The left pane displays the code for a PL/SQL procedure named 'PR_MULTIPLICA'. The right pane shows the 'Ejecutar PL/SQL' dialog, which is used to execute the selected code. The 'Destino' (Destination) is set to 'PR_MULTIPLICA'. The 'Bloque PL/SQL' (PL/SQL Block) contains the code for the procedure. A green arrow points from the 'Ejecutar' button in the top toolbar to the 'Ejecutar PL/SQL' dialog.

Code in the left pane:

```
CREATE OR REPLACE PROCEDURE PR_MULTIPLICA
(
  PI_NUM1 IN NUMBER
, PI_NUM2 IN NUMBER DEFAULT 2
) AS
  v_result number;
BEGIN
  v_result := pi_num1 * pi_num2;
  dbms_output.put_line('Resultado: '||v_result);
END PR_MULTIPLICA;
```

Execution Dialog (Ejecutar PL/SQL):

Destino: PR_MULTIPLICA

Parámetros:

Parámetro	Tipo de Dato	Modo	Valor de Entrada
PI_NUM1	NUMBER	IN	NULL
PI_NUM2	NUMBER	IN	NULL

Bloque PL/SQL:

```
DECLARE
  PI_NUM1 NUMBER;
  PI_NUM2 NUMBER;
BEGIN
  PI_NUM1 := NULL;
  PI_NUM2 := NULL;

  PR_MULTIPLICA(
    PI_NUM1 => PI_NUM1,
    PI_NUM2 => PI_NUM2
  );
  --rollback;
END;
```

Buttons at the bottom: Guardar Archivo..., Desde Archivo..., Restablecer, Ayuda, Aceptar, Cancelar.

The screenshot displays the Oracle SQL Developer environment. On the left, the 'Script' window shows the following PL/SQL code:

```
CREATE OR REPLACE
PROCEDURE PR_LISTA_DEP (PI_DEP_ID IN DEPARTMENT.DEPARTMENT_ID%TYPE DEFAULT 10) AS
    v_nombre      department.name%type;
    v_region      location.regional_group%type;
BEGIN
    SELECT name, regional_group
    INTO v_nombre, v_region
    FROM department D,
         location L
    WHERE L.location_id = D.location_id
           AND D.department_id = PI_DEP_ID;

    dbms_output.put_line(v_nombre||' '||v_region);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        dbms_output.put_line('Error: No existe un departamento con el ID '||PI_DEP_ID||'.');

    WHEN OTHERS THEN
        dbms_output.put_line('Error inesperado: '||sqlerrm);
END PR_LISTA_DEP;
```

On the right, the 'BD_SALES' database is selected. The 'Hoja de Trabajo' (Worksheet) window shows the execution of the procedure with the following SQL:

```
BEGIN
PR_LISTA_DEP(11111111111111);
END;
```

The 'Salida de Script' (Script Output) window shows the execution results:

```
Tarea terminada en 0,019 segundos

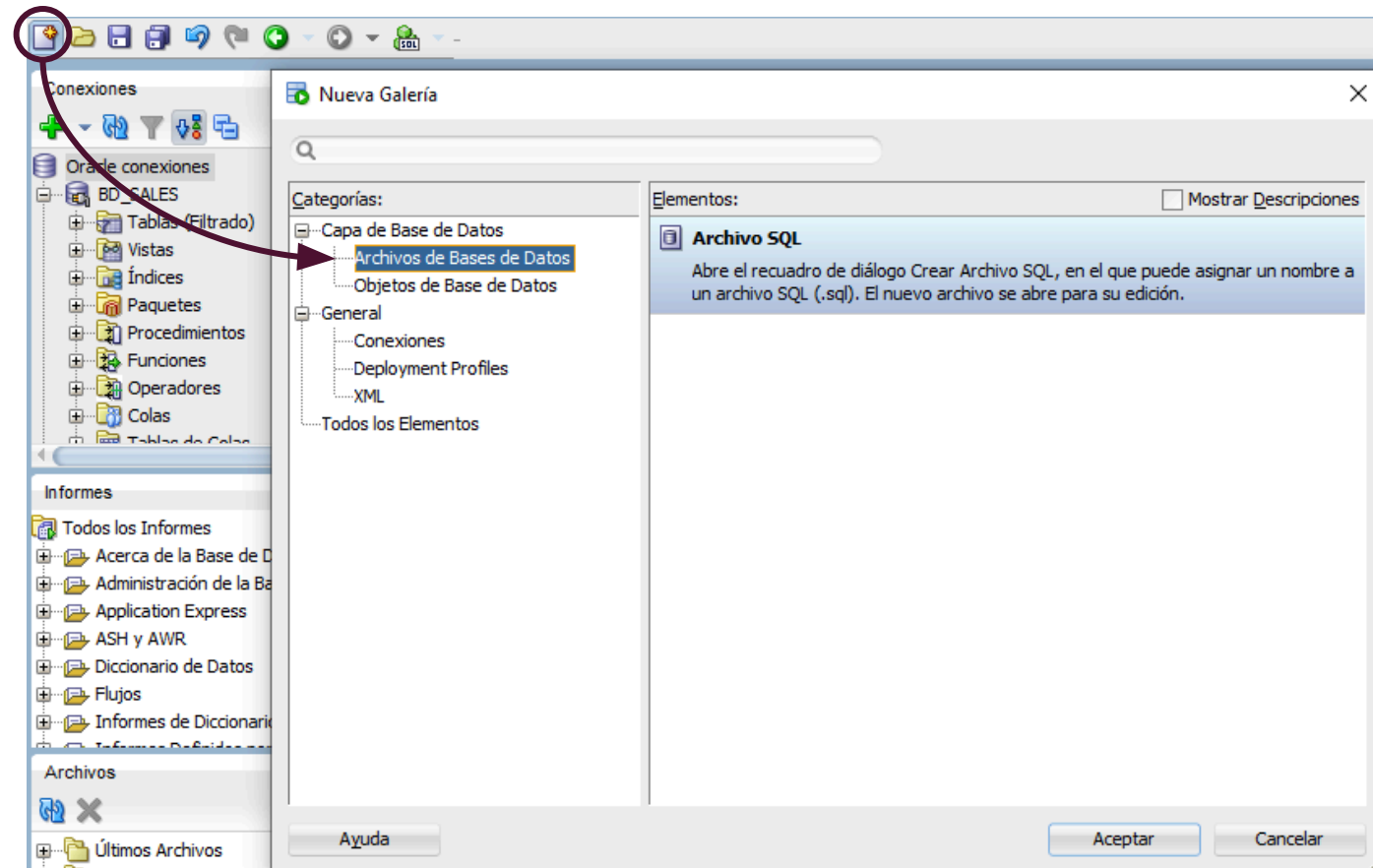
Error: No existe un departamento con el ID 11111111111111.

Procedimiento PL/SQL terminado correctamente.
```

A green arrow points from the 'Error: No existe un departamento con el ID 11111111111111.' message in the output window to the corresponding line in the PL/SQL code, highlighting the error handling logic.

Registro:

Crear un cursor:



Ejemplo: DECLARE

```
CURSOR c_emple IS
  SELECT first_name, last_name
  FROM employee
  WHERE department_id = 10; --PI_DEP_ID;
r_emple c_emple%rowtype;
BEGIN
  OPEN c_emple;
  dbms_output.put_line('a');
  FETCH c_emple INTO r_emple;
  WHILE c_emple%found LOOP
    dbms_output.put_line(r_emple.first_name||' '||r_emple.last_name);
    FETCH c_emple INTO r_emple;
  END LOOP;
  dbms_output.put_line('Hay '||c_emple%rowcount||' empleados');
  CLOSE c_emple;
END;
```

Cuando se implementa un **FOR**, no hace falta abrir, recorrer y cerrar el **Cursor** porque lo hace automáticamente. **Solo hay que declararlo.**

```
FOR R_xx IN C_xx LOOP
  -
END LOOP;
```

Ejemplo de FOR: DECLARE

```
CURSOR c_emple IS
  SELECT first_name, last_name
  FROM employee
  WHERE department_id = 10; --PI_DEP_ID
v_cant number;
BEGIN
  v_cant := 0;
  FOR r_emple IN c_emple LOOP
    dbms_output.put_line(r_emple.first_name||' '||r_emple.last_name);
    v_cant := v_cant + 1;
  END LOOP;
  dbms_output.put_line('Hay '||v_cant||' empleados');
END;
```

Funciones

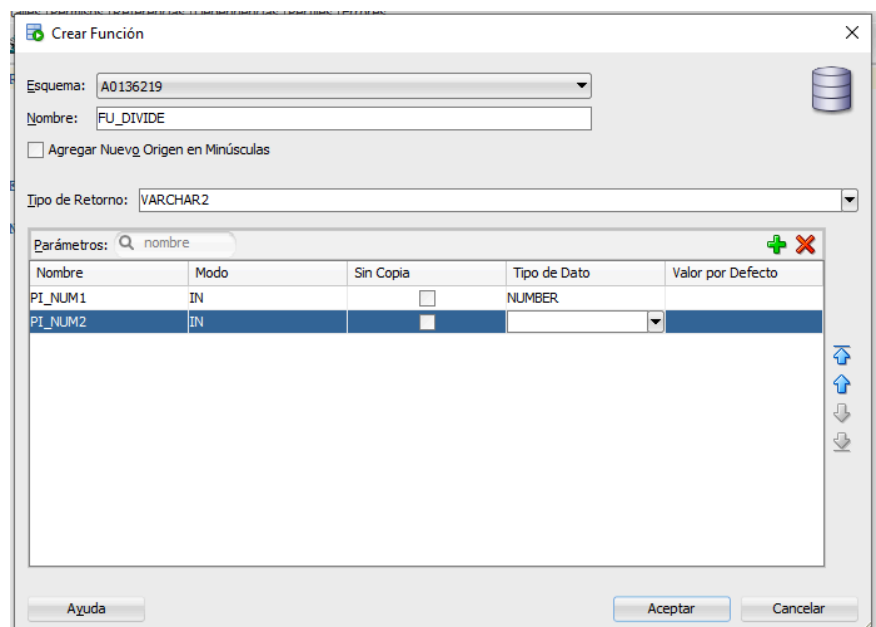
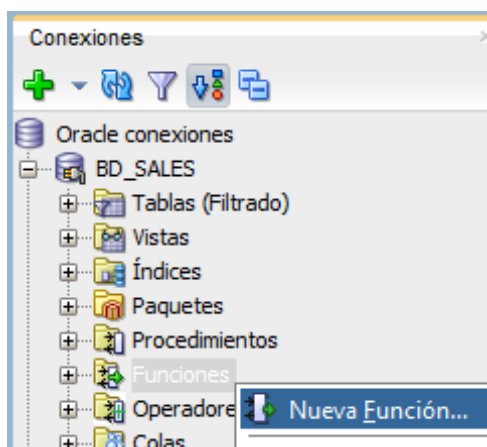
Devuelve un valor en sí mismo o cancela. No termina con un mensaje. ¡PROHIBIDO EL dbms_output!

Estructura: **CREATE OR REPLACE**

FUNCTION FU_nombre (PI_xx tipo) → PARÁMETRO (FU_ = Función)

RETURN tipo IS / AS

Crear una función:



Ejemplo de Función, con trato de errores:

The screenshot shows the SQL Developer interface. On the left, the 'FU_DIVIDE' function is defined with an exception handler for division by zero. On the right, a script is executed, and the 'Resultado de la Consulta' pane shows an error.

```
CREATE OR REPLACE FUNCTION FU_DIVIDE (  
    PI_NUM1 IN NUMBER,  
    PI_NUM2 IN NUMBER)  
    RETURN VARCHAR2 AS  
BEGIN  
    RETURN PI_NUM1 / PI_NUM2;  
  
EXCEPTION  
    WHEN ZERO_DIVIDE THEN  
        RAISE_APPLICATION_ERROR (-20001, 'Falla por división por cero');  
END FU_DIVIDE;
```

Hoja de Trabajo: BD_SALES

```
BEGIN  
    dbms_output.put_line(fu_divide(10/0));  
END;
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0,029 segundos

BEGIN

ERROR en la línea 1:
ORA-20001: Falla por división por cero
ORA-06512: at "A0136219.FU_DIVIDE", line 10
ORA-06512: at line 2

<https://docs.oracle.com/error-help/db/ora-20001/>

More Details :
<https://docs.oracle.com/error-help/db/ora-20001/>
<https://docs.oracle.com/error-help/db/ora-06512/>

Ejemplo de Función, con trato de errores:

The screenshot shows the SQL Developer interface. On the left, the 'FU_EMPLE' function is defined with an exception handler for 'NO DATA FOUND'. On the right, two scripts are shown: one that executes successfully and one that throws an error.

```
CREATE OR REPLACE FUNCTION FU_EMPLE (  
    PI_EMP_ID IN EMPLOYEE.EMPLOYEE_ID%TYPE)  
    RETURN VARCHAR2 AS  
    v_nombre employee.first_name%type;  
    v_apellido employee.last_name%type;  
BEGIN  
    SELECT first_name, last_name  
    INTO v_nombre, v_apellido  
    FROM employee  
    WHERE employee_id = pi_emp_id;  
  
    RETURN v_nombre || ' ' || v_apellido;  
  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        RAISE_APPLICATION_ERROR(-20001, 'No existe el empleado');  
END FU_EMPLE;
```

Cuando existe el empleado

```
BEGIN  
    dbms_output.put_line(fu_emple(7839));  
END;
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0,022 segundos

FRANCIS KING

Procedimiento PL/SQL terminado correctamente.

Cuando no existe el cliente

```
BEGIN  
    dbms_output.put_line(fu_emple(783));  
END;
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0,024 segundos

BEGIN

*

ERROR en la línea 1:
ORA-20001: No existe el empleado
ORA-06512: at "A0136219.FU_EMPLE", line 16
ORA-06512: at line 2

<https://docs.oracle.com/error-help/db/ora-20001/>

More Details :
<https://docs.oracle.com/error-help/db/ora-20001/>
<https://docs.oracle.com/error-help/db/ora-06512/>